

Лабораторный практикум 11. Фролов А.А.

Вывод на экран десятичных и шестнадцатеричных чисел

Первая часть работы

В первой части была написана процедура `Write_word_dec`, которая выводит на экран беззнаковое десятичное число из регистра `DX`.

Для проверки используется число:

```
1  mov dx, 12345
```

После этого вызывается процедура:

```
1  call Write_word_dec
```

На экран должно быть выведено число:

```
1  12345
```

Число в регистре хранится в двоичном виде, поэтому для вывода его в десятичном виде используется деление на `10`. Остаток от деления является очередной цифрой числа. Но цифры получаются справа налево, поэтому они сначала помещаются в стек, а потом извлекаются из него и выводятся уже в правильном порядке

Файл `test.asm`

```
1  .model tiny
2  .code
3  org 100h
4
5  start:
6      mov dx, 12345      ; number for test
7      call Write_word_dec ; print number from DX
8
9      int 20h           ; exit to DOS
10
11 include Video_io.asm
12
13 end start
```

Файл `Video_io.asm`

```

1  ; -----
2  ; Write_char
3  ; Prints one character from DL
4  ; -----
5  Write_char proc
6      mov ah, 02h
7      int 21h
8      ret
9  Write_char endp
10
11
12 ; -----
13 ; Write_digit_hex
14 ; Prints one digit: 0..9 or A..F
15 ; AL = digit
16 ; -----
17 Write_digit_hex proc
18     cmp al, 9
19     jbe digit
20
21     add al, 37h        ; 10 → A, 11 → B ...
22     jmp print
23
24 digit:
25     add al, 30h        ; 0 → '0', 1 → '1' ...
26
27 print:
28     mov dl, al
29     call Write_char
30     ret
31 Write_digit_hex endp
32
33
34 ; -----
35 ; Write_byte_hex
36 ; Prints byte from AL as two HEX digits
37 ; Example: 3F → 3F
38 ; -----
39 Write_byte_hex proc
40     mov bl, al        ; save byte in BL
41
42     shr al, 4         ; high digit
43     call Write_digit_hex
44
45     mov al, bl
46     and al, 0Fh       ; low digit
47     call Write_digit_hex
48
49     ret
50 Write_byte_hex endp

```

```

51
52
53 ; -----
54 ; Write_word_dec
55 ; Prints unsigned decimal number from DX
56 ; Example: DX = 12345 → 12345
57 ; -----
58 Write_word_dec proc
59     mov ax, dx        ; AX = number
60     mov bx, 10         ; divide by 10
61     xor cx, cx         ; CX = digit counter
62
63 divide:
64     xor dx, dx         ; clear DX before division
65     div bx             ; AX = AX / 10, DX = remainder
66
67     push dx            ; save digit in stack
68     inc cx             ; count digit
69
70     or ax, ax          ; check AX == 0
71     jnz divide         ; if not zero, continue
72
73 print_digits:
74     pop ax             ; get digit
75     call Write_digit_hex
76
77     loop print_digits
78
79     ret
80 Write_word_dec endp

```

Описание работы процедуры Write_word_dec

В начале процедуры число из регистра `DX` переносится в `AX`, потому что команда `div` работает с регистром `AX`.

```
1  mov ax, dx
```

Затем в `BX` записывается число `10`, так как нужно получить десятичные цифры:

```
1  mov bx, 10
```

Регистр `CX` используется как счётчик цифр. Он обнуляется командой:

```
1  xor cx, cx
```

Перед каждым делением регистр `DX` очищается:

```
1 xor dx, dx
```

После команды:

```
1 div bx
```

в `AX` находится частное, а в `DX` — остаток от деления. Остаток является очередной цифрой числа, поэтому он помещается в стек:

```
1 push dx
```

Так как цифры получаются в обратном порядке, стек нужен для того, чтобы потом вывести их правильно.

Проверка окончания деления выполняется командой:

```
1 or ax, ax
```

Если `AX` не равен нулю, программа снова переходит к делению:

```
1 jnz divide
```

Когда все цифры получены, они достаются из стека и выводятся на экран.

Сборка и запуск с TASM

Для сборки программы использовались команды:

```
1 tasm Test.asm
2 tlink /t Test.obj, Video_io.com
```

После сборки запускается файл:

```
1 video_io
```

Результат работы программы:

```
1 12345
```

Лабораторная работа

В лабораторной части требуется написать две разные процедуры вывода шестнадцатеричного числа, которое содержится в одном слове, то есть в двух байтах.

Первая процедура работает похожим способом, как процедура вывода десятичного числа: число делится на 16, остатки помещаются в стек, а затем выводятся.

Вторая процедура `Write_word_hex` работает проще. Она выводит слово как два байта: сначала старший байт, затем младший. Для этого два раза вызывается процедура `Write_byte_hex`.

Обновлённый файл `Video_io.asm`

В файл `Video_io.asm` в конец были добавлены две процедуры: `Write_word_hex_1` и `Write_word_hex`.

```
1  ; -----
2  ; Write_char
3  ; Prints one character from DL
4  ; -----
5  Write_char proc
6      mov ah, 02h
7      int 21h
8      ret
9  Write_char endp
10
11
12 ; -----
13 ; Write_digit_hex
14 ; Prints one digit: 0..9 or A..F
15 ; AL = digit
16 ; -----
17 Write_digit_hex proc
18     cmp al, 9
19     jbe digit
20
21     add al, 37h          ; 10 → A, 11 → B ...
22     jmp print
23
24 digit:
25     add al, 30h          ; 0 → '0', 1 → '1' ...
26
27 print:
28     mov dl, al
29     call Write_char
30     ret
31 Write_digit_hex endp
32
33
34 ; -----
```

```

35 ; Write_byte_hex
36 ; Prints byte from AL as two HEX digits
37 ; Example: 3F → 3F
38 ; -----
39 Write_byte_hex proc
40     mov bl, al          ; save byte in BL
41
42     shr al, 4           ; high digit
43     call Write_digit_hex
44
45     mov al, bl
46     and al, 0Fh        ; low digit
47     call Write_digit_hex
48
49     ret
50 Write_byte_hex endp
51
52
53 ; -----
54 ; Write_word_dec
55 ; Prints unsigned decimal number from DX
56 ; Example: DX = 12345 → 12345
57 ; -----
58 Write_word_dec proc
59     mov ax, dx          ; AX = number
60     mov bx, 10          ; divide by 10
61     xor cx, cx          ; CX = digit counter
62
63 divide:
64     xor dx, dx          ; clear DX before division
65     div bx              ; AX = AX / 10, DX = remainder
66
67     push dx             ; save digit in stack
68     inc cx              ; count digit
69
70     or ax, ax           ; check AX == 0
71     jnz divide          ; if not zero, continue
72
73 print_digits:
74     pop ax              ; get digit
75     call Write_digit_hex
76
77     loop print_digits
78
79     ret
80 Write_word_dec endp
81
82
83 ; -----
84 ; Write_word_hex_1

```

```

85 ; Prints word from DX in HEX
86 ; Uses division by 16 and stack
87 ; Example: DX = 1234h → 1234
88 ; Example: DX = 003Fh → 3F
89 ; -----
90 Write_word_hex_1 proc
91     mov ax, dx            ; AX = number
92     mov bx, 16            ; divide by 16
93     xor cx, cx            ; CX = digit counter
94
95 divide_hex:
96     xor dx, dx            ; clear DX before division
97     div bx                ; AX = quotient, DX = remainder
98
99     push dx               ; save digit
100    inc cx                ; count digit
101
102    or ax, ax              ; check AX == 0
103    jnz divide_hex
104
105 print_hex:
106     pop ax                ; get digit
107     call Write_digit_hex
108
109     loop print_hex
110
111     ret
112 Write_word_hex_1 endp
113
114
115 ; -----
116 ; Write_word_hex
117 ; Prints word from DX as 4 HEX digits
118 ; Uses Write_byte_hex two times
119 ; Example: DX = 1234h → 1234
120 ; Example: DX = 003Fh → 003F
121 ; -----
122 Write_word_hex proc
123     mov cx, dx            ; save word in CX
124
125     mov al, ch            ; high byte
126     call Write_byte_hex
127
128     mov al, cl            ; low byte
129     call Write_byte_hex
130
131     ret
132 Write_word_hex endp

```

Описание первой процедуры `Write_word_hex_1`

Процедура `Write_word_hex_1` выводит шестнадцатеричное число из регистра `DX`.

Она работает почти так же, как `Write_word_dec`, только число делится не на 10, а на 16:

```
1  mov bx, 16
```

Остаток от деления на 16 является одной шестнадцатеричной цифрой. Эти цифры также получаются справа налево, поэтому они помещаются в стек.

После этого цифры достаются из стека и выводятся с помощью процедуры:

```
1  call Write_digit_hex
```

Особенность этой процедуры в том, что она не выводит ведущие нули.

Примеры:

```
1  003Fh → 3F
2  000Fh → F
3  1234h → 1234
4  FFFFh → FFFF
```

Описание второй процедуры `Write_word_hex`

Процедура `Write_word_hex` выводит слово как четыре шестнадцатеричных символа.

Сначала число из регистра `DX` сохраняется в `CX`:

```
1  mov cx, dx
```

Затем выводится старший байт:

```
1  mov al, ch
2  call Write_byte_hex
```

После этого выводится младший байт:

```
1  mov al, cl
2  call Write_byte_hex
```

Так как процедура `Write_byte_hex` всегда выводит байт двумя символами, процедура `Write_word_hex` всегда выводит ровно четыре символа.

Примеры:

```
1  003Fh → 003F
2  000Fh → 000F
3  1234h → 1234
4  FFFFh → FFFF
```

Это удобно для вывода адресов памяти, потому что адрес обычно записывается четырьмя шестнадцатеричными цифрами.

Файл test1.asm

Файл test1.asm используется для проверки первой процедуры Write_word_hex_1.

```
1  .model tiny
2  .code
3  org 100h
4
5  start:
6      mov dx, 1234h
7      call Write_word_hex_1
8
9      int 20h
10
11 include Video_io.asm
12
13 end start
```

После запуска программа должна вывести:

```
1  1234
```

Если через Debug заменить число на 003Fh, программа выведет:

```
1  3F
```

Это правильно, потому что первая процедура не выводит нули слева.

Файл test2.asm

Файл test2.asm используется для проверки второй процедуры Write_word_hex.

```
1  .model tiny
2  .code
3  org 100h
4
5  start:
```

```
6      mov dx, 1234h
7      call Write_word_hex
8
9      int 20h
10
11     include Video_io.asm
12
13     end start
```

После запуска программа должна вывести:

```
1      1234
```

Если через Debug заменить число на `003Fh` , программа выведет:

```
1      003F
```

Это показывает главное отличие второй процедуры: она всегда выводит слово полностью, четырьмя символами.

Сборка и запуск первой программы

Для сборки первой тестовой программы используются команды:

```
1      tasm test1.asm
2      tlink /t test1.obj, hex1.com
```

Запуск:

```
1      hex1
```

Ожидаемый результат:

```
1      1234
```

Сборка и запуск второй программы

Для сборки второй тестовой программы используются команды:

```
1      tasm test2.asm
2      tlink /t test2.obj, hex2.com
```

Запуск:

```
1      hex2
```

Ожидаемый результат:

```
1 1234
```

Проверка через Debug

Для проверки можно использовать Debug и менять значение числа по адресам 101h и 102h .

Например, для проверки значения 003Fh :

```
1 debug hex1.com
```

```
1 -e 101 3F 00
2 -g=100
3 -q
```

Для первой процедуры результат будет:

```
1 3F
```

Для второй процедуры:

```
1 003F
```

Для проверки максимального значения FFFFh :

```
1 -e 101 FF FF
2 -g=100
```

Обе процедуры должны вывести:

```
1 FFFF
```

Вывод

В ходе лабораторной работы были изучены способы вывода десятичных и шестнадцатеричных чисел на экран.

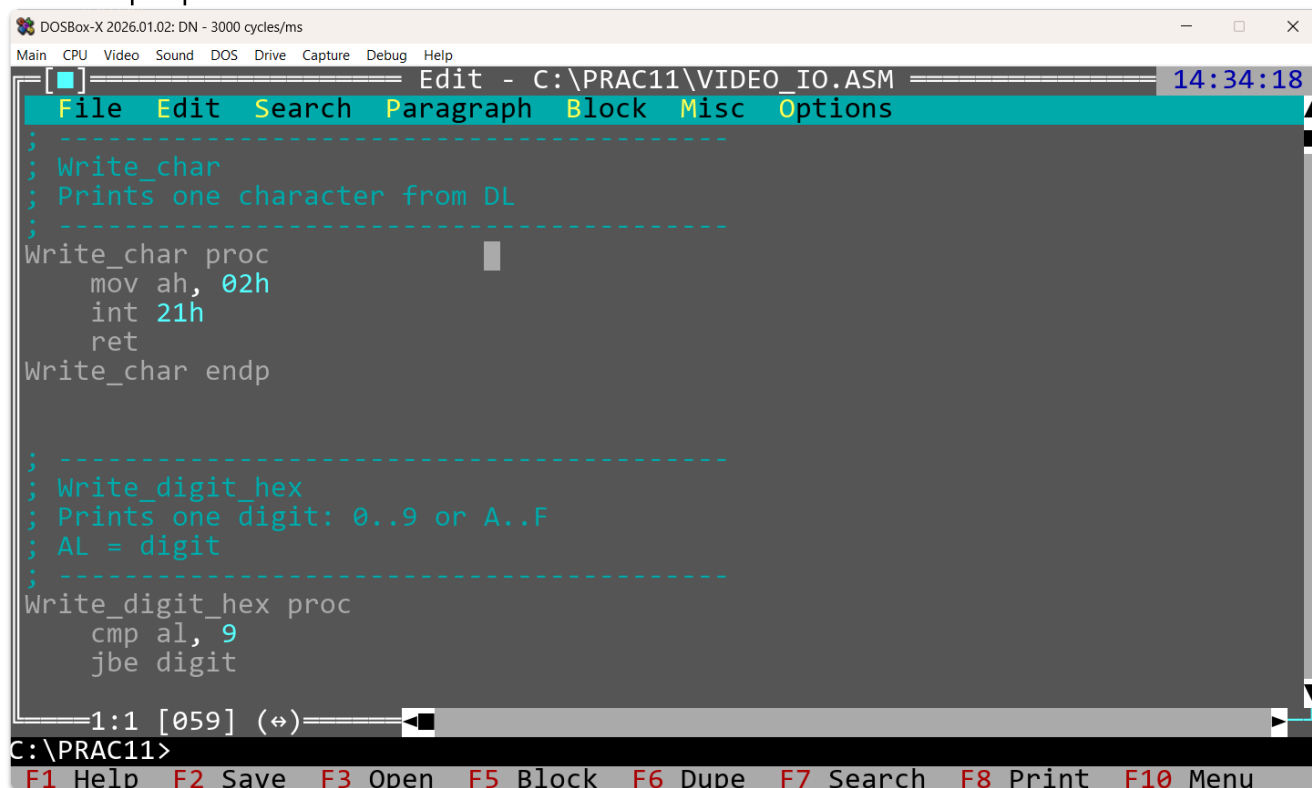
Сначала была написана процедура `Write_word_dec` , которая выводит десятичное число из регистра `DX` . В этой процедуре используется деление на 10 и стек, чтобы вывести цифры в правильном порядке.

Затем были написаны две процедуры для вывода шестнадцатеричного слова. Первая процедура `Write_word_hex_1` использует деление на 16 и стек. Вторая процедура `Write_word_hex` выводит слово проще — через два вызова процедуры `Write_byte_hex` .

В результате были закреплены навыки работы с регистрами, стеком, командой деления `div`, условными переходами и разделением программы на несколько исходных файлов.

Скриншоты

Часть программы:



The screenshot shows a DOSBox-X window with the Turbo Assembler editor open. The title bar reads "DOSBox-X 2026.01.02: DN - 3000 cycles/ms". The menu bar includes "Main", "CPU", "Video", "Sound", "DOS", "Drive", "Capture", "Debug", and "Help". The editor window is titled "Edit - C:\PRAC11\VIDEO_IO.ASM" and shows the following assembly code:

```
; -----  
; Write_char  
; Prints one character from DL  
; -----  
Write_char proc  
    mov ah, 02h  
    int 21h  
    ret  
Write_char endp  
  
; -----  
; Write_digit_hex  
; Prints one digit: 0..9 or A..F  
; AL = digit  
; -----  
Write_digit_hex proc  
    cmp al, 9  
    jbe digit
```

The status bar at the bottom of the editor shows "1:1 [059] (⇌)". Below the editor, a command line shows "C:\PRAC11>". At the very bottom, a row of function key shortcuts is displayed: "F1 Help F2 Save F3 Open F5 Block F6 Dupe F7 Search F8 Print F10 Menu".

Сборка и запуск:

```
C:\PRAC11>tasm test1.asm  
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International  
  
Assembling file:   test1.asm  
Error messages:   None  
Warning messages: None  
Passes:           1  
Remaining memory: 436k  
  
C:\PRAC11>tasm test2.asm  
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International  
  
Assembling file:   test2.asm  
Error messages:   None  
Warning messages: None  
Passes:           1  
Remaining memory: 436k  
  
C:\PRAC11>
```

```
C:\PRAC11>tlink /t test1.obj, hex1.com
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International
C:\PRAC11>tlink /t test2.obj, hex2.com
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International
C:\PRAC11>
```

```
C:\PRAC11>hex1.com
1234
```

```
C:\PRAC11>hex2.com
1234
```

Debug:

```
C:\PRAC11>debug hex1.com
-e 101 3F 00
-g
3F
```

```
C:\PRAC11>debug hex2.com
-e 101 FF 00
-g
00FF
```